Sprite: Bridging the Gap in SpMM Kernel at Low-Level Sparsity for Efficient LLM Inference

Mingge Lu, Jingwei Sun, Junqing Lin, Guangzhong Sun
University of Science and Technology of China
mingge@mail.ustc.edu.cn

***Abstract*—The rapid growth of parameter scale for large language models (LLMs) poses significant challenges for memory and computational costs in deployment. Unstructured pruning is an effective model compression technique with minimal accuracy loss, which introduces fine-grained and low-level sparsity for parameters in weight matrices. However, exploiting unstructured sparse matrix multiplication (SpMM) at low-level sparsity for pruned LLM inference on GPUs remains challenging. Existing solutions leverage bitmap-based compression strategies to achieve memory efficiency, but struggle with substantial computation overheads in decoding irregular non-zero elements on CUDA Cores, thereby wasting the high memory bandwidth and computational throughput of modern GPUs.**

In this paper, we present Sprite, a novel framework that bridges the gap in SpMM kernel to accelerate sparse LLM inference. Sprite introduces Thread-Order Sparse Encoding (ToSparsE), a hardware-aware sparse compression format that arranges irregular non-zero elements into a thread-order sequence, which unlocks the opportunity to reduce bitmap decoding overhead with regular memory layout. Furthermore, Sprite incorporates a buffer-based decoding algorithm and tailored optimizations for the SpMM kernel, further enhancing computational efficiency. Evaluated on the typical sparsity range for LLM pruning (30%-70%), Sprite achieves up to 1.83× speedup over SOTA work SpInfer and 2.26× over cuBLAS at the kernel level on A100, and delivers up to 1.64× end-to-end inference acceleration at 60% sparsity on H200, offering a comprehensive solution for efficient sparse LLM inference.

I. Introduction

Large Language Models (LLMs) [1], [19] have demonstrated remarkable performance across a wide range of downstream tasks in natural language processing, including question answering [41], text generation [30] and agentic applications [3]. However, the ever-increasing parameter scales require substantial memory and computational resources, posing significant challenges for their practical deployment, especially in resource-constrained environments.

To make LLMs more accessible and efficient, considerable efforts have been made to compress these models, including pruning [4], [13], [38], [46], quantization [14], [28], [57], and knowledge distillation [31], [40], [47]. Model pruning, in particular, is a promising approach that removes a portion of less salient parameters in neural networks, introducing sparsity into the model. Among different choices of granularity, unstructured pruning [5], [13], [46], [69] is particularly attractive because it removes individual weights without enforcing structural constraints, thereby offering the most flexibility and preserving model quality better than structured [16], [38], [55], [63] and semi-structured methods [12], [66], [68].

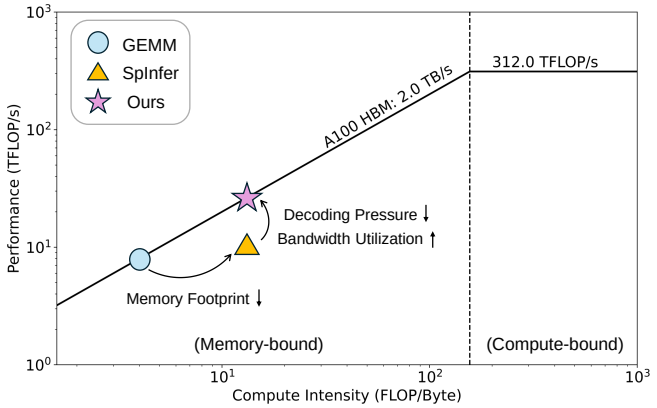


Fig. 1. Roofline comparison of GEMM, SpInfer, and Sprite (Ours). SpInfer reduces memory footprint but struggles to achieve computation efficiency gains. Sprite addresses the bitmap-decoding bottleneck and consequently improves kernel performance with higher memory-bandwidth utilization.

For smaller models such as ResNet [20] that can be pruned with high sparsity levels (over 90%) [21], [37], many previous works [17], [33], [70] propose high-performance sparse matrix multiplication (SpMM) kernels to accelerate sparse model inference. However, LLM pruning typically operates at **low-level sparsity** (around 50%) [13], [46], where traditional methods suffer from large indexing overhead and struggle to enhance SpMM performance. To address this, recent studies propose bitmap-based indexing format [11], [27]. SpInfer [11] is a representative design, showing that bitmap-encoding format can make low-level sparsity practical for accelerating LLM inference. Instead of storing explicit indices for non-zero values, SpInfer records the presence of each element in the sparse matrix with only one bit, which encodes the coordinate information in a memory-efficient manner.

Unfortunately, reducing the memory footprint does not always translate into actual gains in kernel efficiency. Although bitmap encoding reduces global memory traffic, the compressed representation must be decoded before Tensor Core execution, which creates **two key bottlenecks** that limit the benefit of memory savings. First, **index translation imposes heavy computation pressure**. To recover the coordinates of non-zero elements from the bitmap, prior approaches typically count the number of set-bits before each target position, introducing arithmetic overhead for address reconstruction. Second, **bit-wise conditional branches incur substantial**

operation overhead. To unpack values into Tensor Core operands, the SpMM kernel repeatedly inspects the bit-wise bitmap entries and conditionally fetches a non-zero value or inserts zero, resulting in multiple logic and bitwise operations. Above all, the decoding process relies on extensive operations that execute on CUDA Cores, creating high scalar instruction pressure and preventing the kernel from fully utilizing the memory bandwidth of modern GPUs (Figure 1).

To bridge these gaps, this paper presents Sprite, a high-performance SpMM framework specifically designed to overcome the bitmap-decoding obstacles at low-level sparsity through sparse matrix storage and computation kernel co-design. At the core of Sprite is Thread-Order Sparse Encoding (ToSparsE), a novel sparse matrix storage format that reorganizes non-zero values into a thread-oriented order. This hardware-aware design allows GPU threads to consume non-zeros with regular addresses, which eliminates the need for expensive index translation process. To enhance the benefit of ToSparsE format, Sprite uses a tile merging strategy, which reduces the padding overhead for the irregular sparse matrix. On top of the ToSparsE format, Sprite integrates a suite of optimizations for the SpMM kernel, enhancing the utilization of GPU resources: (1) Buffer-based bitmap decoding algorithm exploits an efficient decoding path that mitigates the bit-wise instruction overhead; (2) Efficient data movement instructions reduce pipeline stalls caused by conventional global memory access strategy; (3) Fine-grained software pipeline fully overlaps global memory transfer, decoding and computation.

To validate our approach, we conduct extensive experiments on A100 and H200 GPUs to evaluate the performance from both the kernel level and the end-to-end framework level. At the kernel level, we compare the performance of Sprite-SpMM with SOTA implementations (cuBLAS, SparTA [70], Flash-LLM [54], Coruscant [27], and SpInfer [11]) across the weight matrices of OPT [65], LLaMA3.1 [18] and Qwen3 [59] model series, where Sprite achieves an average speedup of 1.83×, 1.92×, 1.74×, 1.66× and 1.53× respectively at 60% sparsity on A100. We also integrate Sprite into vLLM to conduct end-to-end generation on Qwen3-32B and LLaMA3.1-70B, achieving an increase of generation throughput (tokens/sec) up to 1.64× over dense baseline at 60% sparsity on H200.

In summary, our key contributions are as follows:

- We analyze the limitations of existing SpMM solutions at low-level sparsity, identifying the scalar computation pressure of bitmap-decoding as the key bottleneck.
- We propose Thread-Order Sparse Encoding (ToSparsE) format, which eliminates the need for expensive index translation in bitmap-decoding process.
- We build Sprite-SpMM kernel, which further reduces the bit-wise instruction overhead and enhances GPU resource utilization with tailored optimizations.
- Experimental results show that Sprite achieves substantial speedups at both the kernel and end-to-end framework levels, outperforming state-of-the-art sparse and dense baselines across the typical sparsity range for pruned LLMs (30% to 70%).

TABLE I
MODEL PERFORMANCE (PERPLEXITY↓) UNDER DIFFERENT SPARSITY LEVELS ON LLAMA MODELS PRUNED WITH SPARSEGPT.

Model	Sparsity Level						
	Dense	30%	40%	50%	60%	70%	80%
LLaMA-65B	3.56	3.61	3.94	4.57	5.82	7.64	32.80
LLaMA2-70B	3.12	3.15	3.40	3.98	4.98	6.15	25.86

II. BACKGROUND

A. Unstructured Sparsity in LLMs

Weight pruning is an effective model compression technique that selectively removes less important weights. Existing pruning methods can be broadly categorized into structured [16], [38], [55], [63], semi-structured [12], [66], [68], and unstructured methods [5], [13], [46]. Unstructured pruning removes arbitrary individual weights, offering the highest flexibility and generally preserving model quality better than structured and semi-structured pruning under the same sparsity level. Previous studies on pruning algorithms [13], [46] have demonstrated that large language models can tolerate low-to-moderate levels of unstructured sparsity.

Table I presents the model perplexity (lower is better) evaluated on WikiText-2 dataset for LLaMA models pruned with SparseGPT [13] across different sparsity levels. The perplexity begins to increase noticeably at 40% sparsity and grows rapidly beyond 70% sparsity. On the one hand, pruning at sparsity levels below 30% provides limited benefits due to insufficient compression. On the other hand, recent studies [26], [61] show that sparsity levels around 70% still offer considerable potential, such as enabling adaptive sparsity allocation across different layers to maximize model flexibility. Based on these observations, existing LLM pruning methods typically target a sparsity range of 30% to 70%. However, this range differs significantly from the sparsity patterns commonly addressed by traditional SpMM kernels [17], [33], [70], which introduces significant challenges for designing efficient SpMM kernels for LLM inference.

B. GPU and Tensor Core Unit

Modern GPUs are high-throughput many-core processors designed to execute large numbers of lightweight threads in parallel. A GPU consists of multiple stream multiprocessors (SMs) and a hierarchical memory system. Each SM integrates CUDA Cores, Tensor Cores (TCs), a large register file, and an on-chip memory subsystem in which the L1 cache and shared memory share configurable capacity. All SMs additionally access a unified L2 cache, which helps reduce global-memory access latency and improve effective bandwidth utilization. From a programming perspective, NVIDIA CUDA exposes a three-level execution hierarchy comprising threads, thread blocks, and grids. Programmers specify the number of threads per block and the number of blocks per grid. Threads are scheduled in warps of 32 lanes under the single instruction, multiple threads (SIMT) execution model, while all warps

belonging to the same thread block execute on the same SM. The memory hierarchy includes large-capacity but high-latency global memory, low-latency shared memory for data reuse within a thread block, and registers that provide the fastest but most limited storage within a thread.

Tensor Cores (TCs), introduced since the Volta microarchitecture, are specialized units designed to accelerate dense matrix multiplication. Different from scalar computation on CUDA cores, TC provides tile-based matrix-matrix computation primitives on register fragments, which can deliver more than 10× throughput improvement. In particular, TC supports the compute primitive of $D_{frag} = A_{frag} \times B_{frag} + C_{frag}$, where $A_{frag} \in \mathbb{R}^{m \times k}$ and $B_{frag} \in \mathbb{R}^{k \times n}$ are inputs, $C_{frag} \in \mathbb{R}^{m \times n}$ is the accumulator, and $D_{frag} \in \mathbb{R}^{m \times n}$ is the output. For FP16 computation, TC operates via warp-level mma (Matrix Multiply Accumulate) instructions, which require matrix shapes $m \times n \times k$ of 16×8×16 or 16×8×8.

Different from the CUDA Cores that operate at the thread level, TC supports the dense mma at the warp level. Before calling TCs, all threads in a warp need to collaboratively load the dense matrix tiles into the Register Fragment operand. The fundamental mismatch between irregular unstructured sparsity and the architecture of TCs poses significant challenge for accelerating unstructured SpMM kernel on modern GPUs.

C. SpMM Kernel for Sparse LLM Inference

LLMs are built upon the Transformer architecture [48], where both self-attention and feed-forward networks contain linear layers that rely on weight projection operations. SpMM serves as the primary computational kernel for linear layers where model weights have been pruned to reduce memory footprint. For a linear layer, we denote the sparse weight matrix as $W \in \mathbb{R}^{M \times K}$, and the dense input activation matrix as $X \in \mathbb{R}^{K \times N}$, where M is the output dimension, K is the hidden dimension, and N is the number of input tokens processed in the current generation step. The linear layer computes the sparse-dense matrix multiplication, yielding the dense output matrix as $Y = W \times X$.

LLM inference contains two phases. In the prefill phase, all tokens in the input sequence are processed in parallel, so N is the input sequence length multiplied by the batch size ($seq_len \times BS$). In the decode phase, tokens are generated in an autoregressive manner, and each generation step processes only one new token at a time, so N is equal to the batch size ($1 \times BS$). The end-to-end generation is dominated by the decode phase, where the matrix multiplications are memory-bound as the N dimension (i.e. batch size) is significantly smaller than the weight dimensions M and K. Therefore, reducing the memory footprint for sparse matrix is critical in accelerating SpMM kernel for LLM inference.

To implement high-performance SpMM kernels for sparse LLMs, Flash-LLM [54] employs the Load-as-Sparse, Compute-as-Dense approach [54] to reduce the memory footprint, where the sparse matrix is loaded from global memory with compressed storage, decoded into register fragment, and then multiplied with the dense matrix using Tensor Cores.

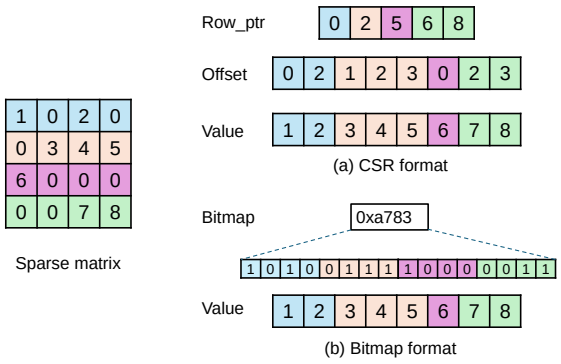


Fig. 2. Comparison of CSR and bitmap compression format for an unstructured sparse matrix. CSR stores non-zero values together with row pointers and explicit offsets, whereas bitmap encoding uses one bit per matrix position to indicate the non-zero value existence.

Since Flash-LLM still focuses on high sparsity levels, it introduces significant storage overheads at low sparsity levels, where the space needed for indices can neutralize pruning's storage gains. While previous methods [50], [54] focus on high sparsity regime, recent works [11], [27] have targeted the practical low-level sparsity range for LLMs. These methods employ bitmap-based compression format to reduce indexing overhead for unstructured sparse matrices.

Bitmap compression can be considered as an existence-based sparse encoding strategy, in contrast to offset-based format like CSR. We illustrate the CSR and Bitmap compression formats in Figure 2. In CSR-style format, each non-zero value is stored with an explicit offset integer. This design enables direct addressing, but its index costs for non-zero elements become significant at low-level sparsity. Bitmap encoding instead uses only one bit to describe whether each possible position in the tile contains a non-zero value. Therefore, bitmap compression avoids storing explicit coordinates for every non-zero element, which is beneficial for low sparsity regime.

III. Motivation

The SOTA work SpInfer [11] shows that bitmap encoding is a promising direction for low-sparsity memory-bound SpMM as it avoids expensive indices and reduces memory footprint significantly. However, decoding the compact bitmap format creates high scalar instruction pressure and prevents the kernel from fully utilizing the memory bandwidth (Figure 3). This leads to our key observations.

Observation 1: Compact storage shifts indexing overhead to runtime computation. In SpInfer's compression format, non-zero values are stored contiguously in warp-level fragment order. Since this layout is compact, it does not provide each thread with a direct address for the values it needs. As illustrated in Figure 4(a), before loading a value, each thread must first count the number of non-zeros preceding its target position in the bitmap. This repeated offset calculation imposes substantial scalar-computation pressure on the CUDA

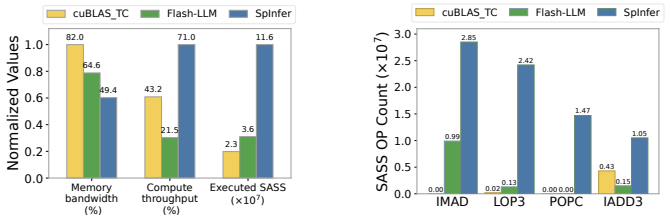


Fig. 3. Micro-level profiling and instruction breakdown comparison of SpInfer against cuBLAS_TC and Flash-LLM on A100 GPU (M/K/N/Sparsity=28k/7k/16/50%, typical in LLM inference).

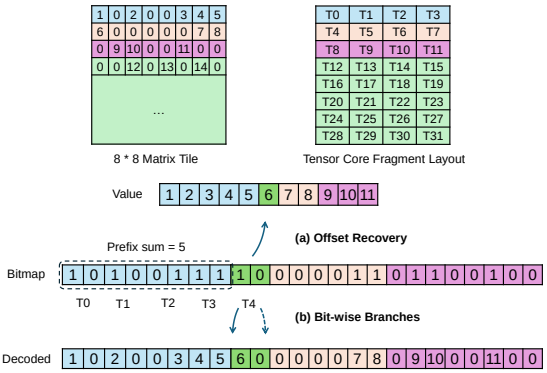


Fig. 4. Bitmap-decoding bottlenecks in SpInfer.

cores, preventing the kernel from fully exploiting the global-memory bandwidth.

A natural design opportunity is to assign each thread a private stream of non-zero values, allowing it to consume its values sequentially without repeatedly recovering global offsets from the bitmap. However, directly storing thread-owned streams in the compressed representation leads to irregular shared-memory accesses because different threads may own different numbers of non-zeros under arbitrary unstructured sparsity. Such imbalance can cause severe shared-memory bank conflicts. Therefore, eliminating online offset recovery requires not only a thread-local value organization, but also an ordered memory layout that enables conflict-free shared-memory access.

Observation 2: Bit-wise decoding introduces substantial instruction overhead. Bitmap decoding requires each thread to make bitmap-dependent decisions. As illustrated in Figure 4(b), for every candidate element in a Tensor Core fragment, a thread must inspect the corresponding bitmap bit to determine whether to load a non-zero value from shared memory or directly write zero into the operand register. Under arbitrary unstructured sparsity, different threads within a warp may observe different bitmap bits, resulting in warp divergence that hurts GPU performance.

To address the control-flow divergence, SpInfer uses the conditional operator $x = condition\ ?\ a : b$, which is typically compiled into a PTX `selp` instruction. This approach prepares both the true operand a and the false operand b, and then selects the result according to the condition. Consequently, for each bitmap bit, SpInfer must load a candidate non-zero value from shared memory and execute a conditional selection. Compared with dense GEMM, this decoding stage does not reduce shared-memory traffic and further introduces additional logical and selection instructions.

A natural design opportunity is to reorganize the bitmap and compressed values into a thread-owned order and perform decoding entirely within thread-local registers. Rather than repeatedly testing bitmap bits and gathering individual values from shared memory, the kernel can first transfer compressed values into thread-local register buffers through regular and conflict-free shared-memory accesses. This organization improves the efficiency of the shared-memory data path. Once the thread-owned values and bitmaps are staged in registers, repeated bit-wise selections can be replaced with NVIDIA GPU's integer instructions such as `__clz`, which efficiently identifies the position of the first set bit in the bitmap. In this way, the kernel can substantially reduce the instruction overhead associated with repeated bit-wise decoding.

IV. Sprite Design

In this section, we present the design and implementation of Sprite, a storage-kernel co-design that targets the performance bottlenecks of SpMM kernels at the low-level sparsity commonly found in pruned LLMs. Its design consists of two key components. First, Sprite introduces Thread-Order Sparse Encoding (ToSparsE), which reorganizes bitmap metadata and non-zero values according to the order in which GPU threads consume them. This regular thread-oriented layout eliminates repeated offset reconstruction from compact bitmaps. Second, Sprite employs a buffer-based decoding algorithm that stages sparse values in thread-owned register queues and reconstructs Tensor Core operands using lightweight register-level operations. Sprite further combines architecture-aware data movement with a hierarchical software pipeline to overlap global-memory transfer, sparse decoding, and Tensor Core computation, enhancing the utilization of GPU resources.

A. ToSparsE: Thread-Order Sparse Encoding

The Sprite framework's foundation lies in its Thread-Order Sparse Encoding (ToSparsE) data structure. ToSparsE is designed to satisfy two competing requirements. First, it must retain the storage efficiency of bitmap compression for arbitrary unstructured sparsity, particularly within the 30%–70% sparsity range targeted by LLM pruning. Second, it must expose a regular access pattern to GPU threads so that compressed weights can be transformed into Tensor Core operands with minimal scalar instructions.

1) Hierarchical Tiling Design: As shown in Figure 6, ToSparsE adopts a four-level hierarchical tiling scheme that partitions the sparse weight matrix from coarse scheduling granularity down to Tensor Core fragment granularity. From the outermost to the finest level, the hierarchy consists of BlockTile, KTile, TensorCoreTile, and FragTile, aligning the sparse representation with thread-block scheduling, Tensor Core computation, and register operand construction.

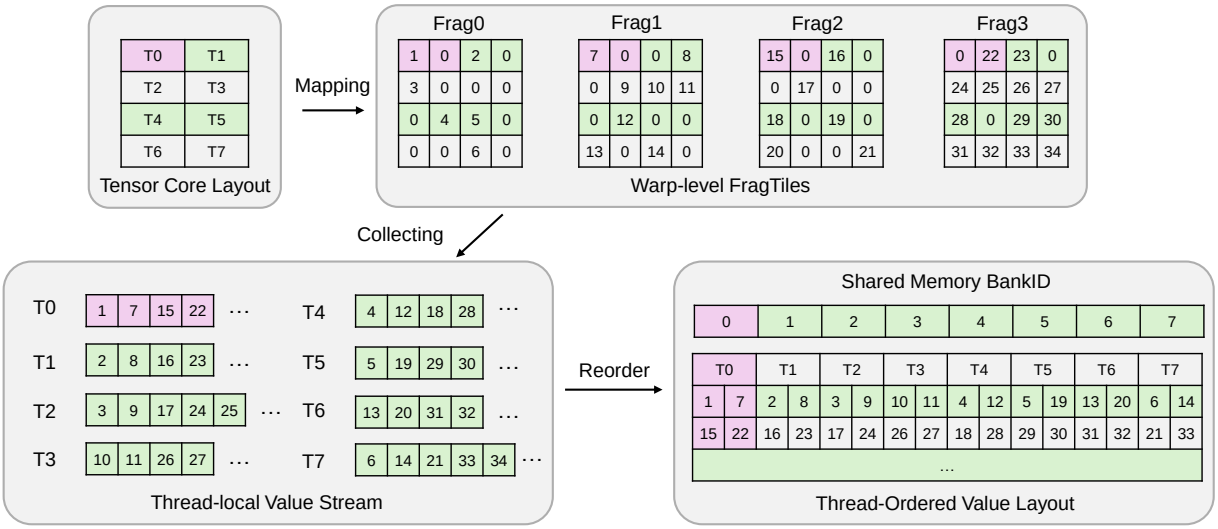


Fig. 5. Overview of the thread-ordered sparse layout in ToSparsE. FragTile is actually 8×8, shown as 4×4 for illustration. non-zero values from warp-level FragTiles are assigned to their corresponding threads in Tensor Core Fragment, collected into thread-local streams, and interleaved by thread order into a bank-striped value layout. Each 32-bit bank contains two non-zero values under FP16 precision.

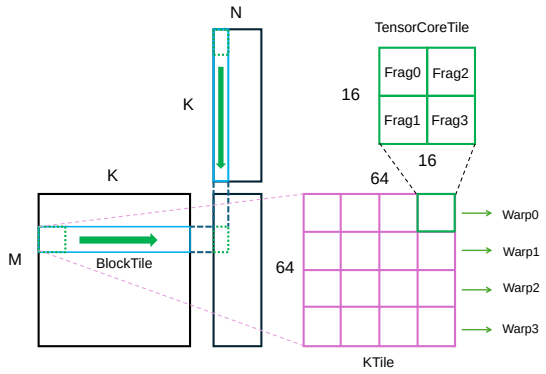


Fig. 6. The tiling design in Sprite's compression format.

❶ The outermost unit is the BlockTile, which covers a $64 \times K$ region of the sparse weight matrix. A BlockTile defines the persistent row region assigned to a CUDA thread block and spans the complete reduction dimension. ❷ Along the K dimension, each BlockTile is partitioned into 64×64 KTiles. A KTile is the basic unit of global-to-shared data movement and pipelined computation, providing a bounded sparse region that can be loaded, decoded, and consumed during one outer-loop iteration. ❸ Each KTile is further decomposed into 16×16 TensorCoreTiles. This shape matches the sparse-weight operand consumed by the `mma.m16n8k16` instruction. Aligning the representation with this granularity groups the bitmap metadata and non-zero values required by each MMA operation and allows the decoder to construct the expected register layout directly. ❹ At the finest level, each TensorCoreTile contains four 8×8 FragTiles. Within each FragTile, matrix positions are assigned to threads according to the Tensor Core operand mapping, with each warp thread responsible for two positions. The four FragTiles are traversed in column-major order, matching the order in which the corresponding operand fragments are constructed and consumed. The hierarchy therefore provides a direct correspondence between sparse matrix positions, execution threads, and Tensor Core operand registers.

2) Sparse Data Layout: As illustrated in Figure 5, ToSparsE reorganizes non-zero values according to the order in which GPU threads consume them. For each FragTile, the offline encoder first determines which thread will consume each non-zero value during fragment construction. This assignment follows the Tensor Core operand layout and the register mapping used by the MMA instruction. After this mapping process is finished, the bitmaps and non-zero values assigned to the same thread are collected into a thread-local stream. Each stream therefore contains exactly the values that a particular thread will load and insert into its operand fragment, in the same temporal order as the kernel consumes them.

A straightforward implementation would store the thread-local streams consecutively, for example placing the entire stream of thread 0 before the stream of thread 1. However, such a layout is inefficient for warp-level execution. Since all threads advance together during sparse loading, consecutive storage of full streams would make parallel threads access memory locations with large and irregular gaps. It also creates unfavorable shared-memory access patterns: threads may repeatedly target the same memory bank or produce non-uniform address strides, leading to bank conflicts and serialization.

ToSparsE instead adopts a bank-striped sparse value layout. The thread-local streams are interleaved across threads by

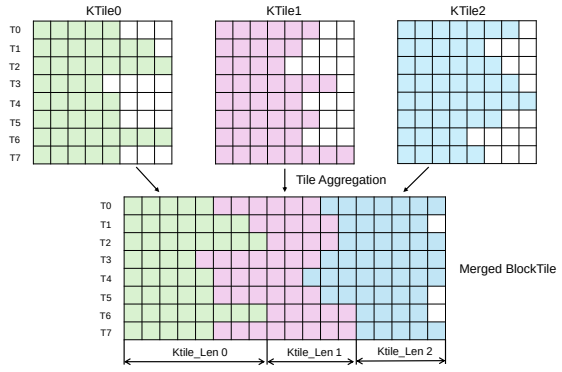


Fig. 7. Illustration of Tile Merging Strategy in ToSparsE. We concatenate thread-local streams across all KTiles in a BlockTile and apply padding after this aggregation, which amortizes the imbalance over the reduction dimension.

consumption round. Each 32-bit bank contains two non-zero values under FP16 precision. In each round, the layout stores the next two values required by thread 0, followed by the next two values required by thread 1, and so on across the warp. As a result, the physical layout follows the warp execution order rather than the matrix coordinate order. At runtime, a thread can derive the address of its next sparse value directly from its thread identifier and the current loading round, without performing bitmap prefix counting or set-bit traversal.

3) Tile Merging Strategy: The thread-order layout of ToSparsE requires every thread to contribute a value to each stored round. Under arbitrary unstructured sparsity, however, different threads receive different numbers of non-zeros. Padding is therefore required for thread streams shorter than the longest stream in the same encoding tile. Applying padding independently at every KTile boundary would accumulate this imbalance across the matrix and substantially weaken the storage advantage of bitmap compression.

As illustrated in Figure 7, ToSparsE merges thread-local streams across all KTiles in a BlockTile and applies padding only after this aggregation. A short stream in one KTile can therefore be compensated by a longer stream in another KTile, and only the final residual difference among the aggregated streams must be padded. This BlockTile-level strategy amortizes thread imbalance over the reduction dimension and minimizes the additional storage introduced by bank striping. For each KTile, the loading length is determined offline by the maximum cumulative nonzero count across threads. The increment over the preceding KTile determines how many values to load, which is collected during preprocessing.

4) Metadata Representation: ToSparsE employs three metadata arrays to represent sparse weight matrices efficiently. ❶ BlockTile_Offset records the starting address of each BlockTile in the compressed value array, enabling direct access to variable-length sparse data. ❷ KTile_Len specifies the loading length for each KTile, computed as the difference between the maximum cumulative nonzero counts of consecutive KTiles. ❸ Thread_Bitmap encodes the sparsity pattern of elements assigned to each thread, indicating the nonzero positions in the corresponding Tensor Core fragments.

B. SpMM Kernel Design of Sprite

Sprite implements a Tensor-Core-based SpMM kernel tailored to the ToSparsE representation. The overview is illustrated in Figure 8. Compressed values are first transferred from global memory to shared memory, then moved from the bank-striped shared-memory layout into thread-owned register buffers, and finally decoded from these buffers into TC operand registers for computation. This organization separates high-bandwidth data movement from fine-grained fragment reconstruction and allows each stage to use the memory-access pattern best suited to its position in the hierarchy.

1) Asynchronous Global Memory Access: Asynchronous global memory access enables the global-to-shared memory transfer to overlap with subsequent computation, which is essential for improving memory bandwidth utilization. On Ampere GPUs, Sprite uses 128-bit asynchronous global-to-shared copies, realized through `LDGSTS.128` instruction. These vectorized transfers move compressed data directly into shared memory without intermediate staging of data through the L1 cache and register file, reducing register-file traffic and the instruction overhead associated with conventional load-and-store sequences. Across participating threads for the `LDGSTS.128` instruction, the vectorized accesses form coalesced global-memory transactions.

To fulfill the potential of Sprite-SpMM kernel in modern GPU generations, we further introduce Warp-Specialized Producer-Consumer Model [56] for Hopper architecture. Specifically, producer warps load data from global memory into shared memory, while consumer warps asynchronously process the previously loaded tiles through shared-to-register transfer, sparse decoding, and Tensor Core computation. The global-to-shared memory transfer leverages asynchronous TMA (Tensor Memory Accelerator) instructions for efficiency, which reduces the overhead of instruction stalls associated with multiple `LDGSTS` instructions.

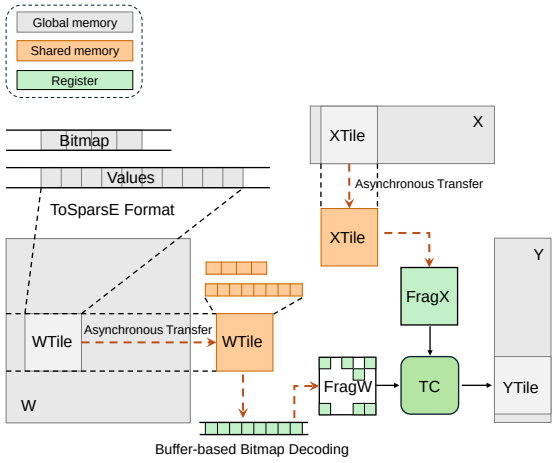


Fig. 8. Overview of Sprite-SpMM kernel design.

Algorithm 1 Thread-Level Buffer-Based Bitmap Decoding

Input: Value array V with length L in shared memory, thread-local bitmap b, register buffer Q_t, thread ID t
Output: TC fragment registers R_t for thread t

// Shared memory load with regular address
for $i \leftarrow 0$ to $L/32 - 1$ **do**
  $x \leftarrow V[i * 32 + t]$ *// Load two FP16 values as 32-bit*
  Q_t.enqueue(x)
end for
// Construct Tensor Core fragment
$R_t \leftarrow \{0\}$ *// Initialize the fragment registers*
$p \leftarrow 0$ *// Initialize the non-zero position pointer*
for $i \leftarrow 0$ to 31 **do**
  $p \leftarrow$ **clz**(b)
  if $p = 32$ **then** *// No remaining set bits*
    break
  end if
  $R_t[p] \leftarrow Q_t$.dequeue() *// Pop one FP16 value*
  // Clear the leading set bit in bitmap
  $mask \leftarrow (1 \ll (31 - p))$
  $b \leftarrow b \ \& \sim mask$
end for

2) Buffer-Based Decoding Algorithm: ToSparsE applies padding only at the BlockTile level, so the thread-local compressed-value stream is continuous across multiple KTiles. Consequently, some values loaded in the current iteration might be required by a subsequent iteration. To address this mismatch between storage granularity and decoding boundary, Sprite introduces a thread-owned register buffer organized as a queue, containing values from its ToSparsE stream. The efficient decoding process is presented in Algorithm 1. During shared memory loading, the thread moves two FP16 values as 32-bit into this queue with regular address in each round. During fragment construction, it consumes values from the head of the queue and places them into the register positions indicated by the corresponding bitmap. This queue-based design provides a clean separation between where values are stored and when values are consumed. The physical sparse layout follows the outermost BlockTile stream to minimize padding, while the compute loop consumes values at the KTile granularity. If a value loaded with the current KTile is not needed immediately, it remains in the thread's register queue and is carried into the following iteration for decoding.

The register capacity requirement is determined offline during preprocessing. Specifically, we simulate the per-thread queue size when traversing all KTiles, tracking the number of nonzero values inserted and consumed from each thread's queue at every iteration. The maximum size across all threads and iterations determines the required register capacity, avoiding overflow while minimizing register overhead. The loading schedule ensures that the cumulative number of loaded values is never smaller than the number of consumed values, eliminating the possibility of underflow.

After values enter the buffer queue, decoding into Tensor Core fragment is performed entirely in registers. Rather than examining every bitmap bit and executing a conditional selection for every candidate position, Sprite uses the NVIDIA GPU's integer intrinsic `__clz` (count leading zeros) to derive the position of first non-zero in the bitmap. First, we pop one value from the register queue and insert it into the corresponding operand position derived by `__clz` instruction. After that, we clear the selected bit, and continue this decoding process until no set bits remain. Since this decoding path performs work only for non-zero positions, it removes the shared-memory load and conditional-selection sequence required for every bitmap bit in conventional bitmap decoding techniques.

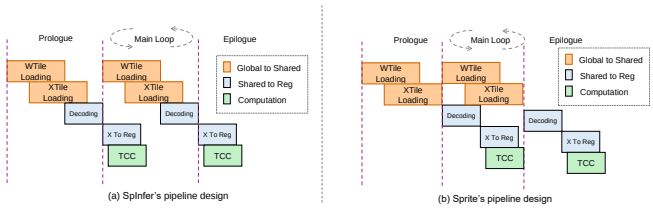


Fig. 9. Pipeline comparison between SpInfer and Sprite.

3) Fine-grained Software Pipeline: Sprite employs a hierarchical two-level software pipeline to overlap global-memory transfer, shared-to-register movement, register-level decoding, and Tensor Core execution. ❶ Inter-tile overlapping. At the coarse level, while the compute stage consumes KTile i from one shared-memory buffer, KTile $i + 1$ is asynchronously prefetched into the alternate buffer. After the current tile has been consumed and the next transfer has completed, the two buffers exchange roles. ❷ Intra-tile overlapping. At the fine level, Sprite further divides each dense tile into finer slices and interleaves register loading with Tensor Core execution.

As shown in Figure 9, SpInfer performs global-memory loading and shared-memory decoding of sparse tiles within the same loop iteration, limiting the opportunity to fully utilize global-memory bandwidth. In contrast, Sprite defers shared-memory decoding to the subsequent iteration, enabling separation and overlap between memory access stage and computation stage. This design hides the computational overhead of decoding under global-memory transfer, shifting the global-memory access back as the dominant stage.

V. Evaluation

We evaluate the performance of Sprite at two levels: the SpMM kernel level and the end-to-end system level. The experiments are conducted on two platforms. ❶ Intel Xeon w5-2455X CPU (3.20GHz) with 2 NVIDIA A100 PCIe 80GB GPUs (Ampere, Compute Capability 8.0). ❷ Intel Xeon PLATINUM 8558P CPU (2.70GHz) with 2 NVIDIA H200 NVL 141GB GPUs (Hopper, Compute Capability 9.0). The code is compiled using GCC 9.4.0 and NVCC 12.3. For kernel-level evaluation, we perform 1000 warm-up iterations followed by 1,000 benchmark executions. For end-to-end evaluation, we repeat model inference 10 times under each configuration.

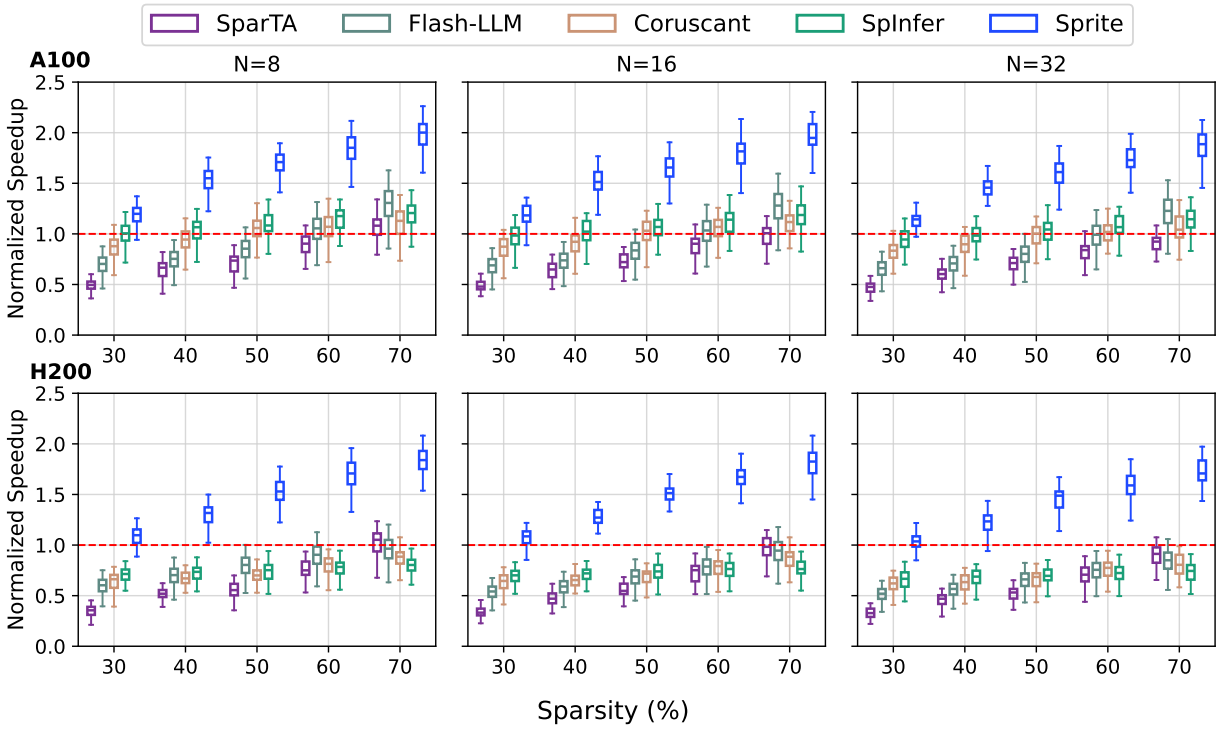


Fig. 10. SpMM kernel performance comparison on A100 and H200 GPUs. N denotes the batch size.

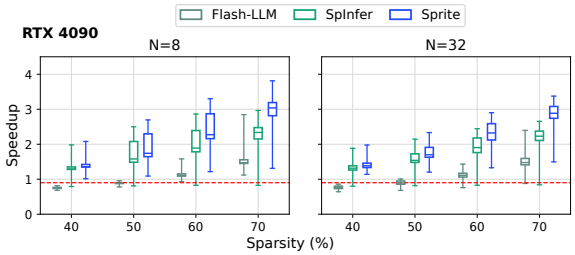


Fig. 11. SpMM kernel performance comparison on RTX4090.

A. Kernel Performance

Dataset of matrices. We evaluate the kernel performance on a broad range of pruned weight matrices in linear layers from prominent LLM series, including OPT (13B, 30B, 66B, and 175B) [65], LLaMA3.1 (8B, 70B and 405B) [18], Qwen3 (8B, 14B and 32B) [59] model. We leverage two popular LLM pruning algorithms Wanda [46] and SparseGPT [13] to demonstrate the effectiveness of our method over different sparsity patterns.

Baselines. We compare Sprite SpMM kernel with several key baselines, including: ❶ Tensor-Core-based cuBLAS, the GEMM baseline used in dense LLM inference frameworks; ❷ SparTA [70], a classic method that leverages Sparse Tensor Cores for unstructured SpMM acceleration; ❸ Flash-LLM [54], the first approach to accelerate high-sparsity LLM inference with Tensor-Core-based SpMM kernel; ❹ Coruscant [27], a hardware-software co-design method for low-sparsity LLM inference, which is optimized on both GPUs and customized hardware; ❺ SpInfer [11], the state-of-the-art bitmap-based SpMM targeting sparse LLM inference at low-level sparsity. Both Coruscant and SpInfer reuse the kernel evaluation framework originally developed by Flash-LLM. We adopt the same framework to ensure fair comparisons under consistent and appropriate settings. Specifically, all methods use the same Split-K configurations.

Results. Figure 10 shows the kernel performance of Sprite SpMM and the baselines on A100 and H200 under 30%, 40%, 50%, 60% and 70% sparsity, the typical range for unstructured LLM pruning. We report the normalized speedup relative to Tensor-Core-Based cuBLAS_TC across all evaluated workloads. The cuBLAS_TC baseline is marked with a dashed line. Sprite consistently outperforms the sparse baselines across the 30%-70% sparsity range, and remains the only method outperforming cuBLAS_TC in most cases.

On A100 GPU, Sprite achieves an average speedup of 1.73× over cuBLAS, and 2.31×, 1.92×, 1.65× and 1.51× over SparTA, Flash-LLM, Coruscant and SpInfer, respectively. At 50% sparsity, SparTA and Flash-LLM generally fail to outperform cuBLAS_TC because their compression formats incur substantial indexing overhead and can produce memory footprints larger than that of the dense representation. In contrast, the bitmap-based methods, Coruscant, SpInfer, and Sprite, benefit from more compact sparse representations. Coruscant nevertheless underperforms SpInfer because its un-

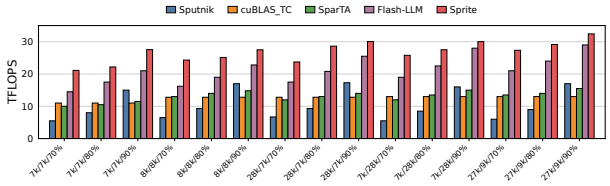


Fig. 12. Kernel Performance at high sparsity levels.

coalesced global-memory accesses limit effective bandwidth utilization. Moreover, memory compression is insufficient to fully exploit sparsity because runtime decoding overhead can offset the reduction in memory traffic. Sprite addresses both problems through its regular thread-ordered representation and efficient decoding pipeline, thereby delivering meaningful speedups over both dense and sparse baselines.

On H200 GPU, Sprite improves over cuBLAS by 1.48× on average and exceeds SparTA, Flash-LLM, Coruscant and SpInfer by 2.47×, 1.97×, 2.11× and 1.96×, respectively. Most sparse baselines underperform cuBLAS_TC on the H200 because conventional global-memory access strategies introduce pipeline stalls and fail to utilize the substantially higher memory bandwidth of the Hopper architecture. Sprite combines its efficient storage and decoding mechanisms with TMA and warp-specialized pipelining optimizations, allowing it to maintain high performance across the evaluated workloads. As shown in Figure 11, we further perform evaluation on the consumer-grade RTX4090, where Sprite achieves an average speedup of 2.03×, 1.43× and 1.21× over cuBLAS, Flash-LLM and SpInfer. In Figure 12, we compare the kernel performance at 70% to 90% sparsity levels. Sprite consistently outperforms Flash-LLM with an average speedup of 1.13×. The kernel efficiency at high sparsity levels stems from the lightweight decoding algorithm, which reduces the decoding computation as the sparsity level increases.

In Figure 13, we evaluate Sprite under different N settings. Sprite is optimized for the memory-bound stage, which achieves up to 1.82× speedup over the dense baseline, and surpasses SpInfer and cuSPARSELt for small N settings. For N smaller than 8, the latency for TC-based methods does not change since the minimum N-size for TC MMA instruction is 8. Note that TC-based methods still outperform the CUDA-Core-based cuBLAS for N smaller than 8. For the compute-bound prefill stage, Sprite underperforms cuBLAS_TC since

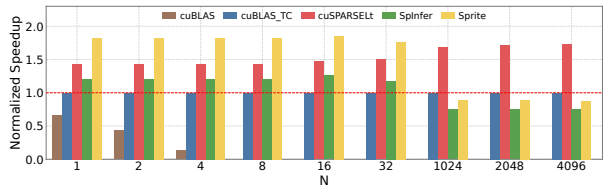


Fig. 13. Kernel performance under different N settings (M/K/Sparsity=28k/7k/50%). cuSPARSELt is conducted on 2:4 sparsity pattern, while other methods are evaluated on unstructured sparsity.

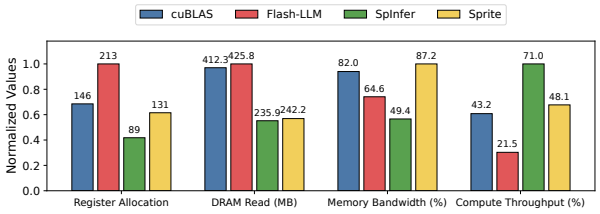


Fig. 14. Micro-level comparison of Sprite against cuBLAS, Flash-LLM, and SpInfer across key metrics with NCU profiler on A100 GPU (M/K/N/Sparsity=28k/7k/16/50%, typical in LLM inference).

there is no skip of computations on dense TC, while cuSPARSELt achieves 1.73× speedup over the dense baseline.

Micro-level Profiling. To provide a detailed understanding for the performance gains of Sprite, we augment our evaluation with micro-level hardware analysis for a representative workload with M/K/N=28k/7k/16 at 50% sparsity. As shown in Figure 14, we collected key metrics including register allocation, DRAM read, memory bandwidth utilization, and compute throughput, using the NVIDIA Nsight Compute profiler.

SpInfer and Sprite leverage vectorized memory transfers that move compressed data directly into shared memory, exhibiting lower register consumption than Flash-LLM. The lower per-thread register usage enables more threads to run concurrently, thereby improving GPU occupancy and boosting overall throughput. The additional registers relative to SpInfer are used by Sprite's register-buffered decoding mechanism. We show the distribution of queue capacity requirements in Figure 15. Overall, the queue capacity requirement becomes smaller as the sparsity increases. The largest queue size is 62, with total register allocation of 146. We further report the occupancy and active warps per SM metrics on representative configurations in Figure 16.

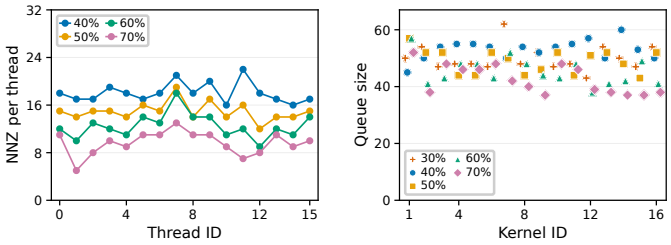


Fig. 15. Inter-thread NNZ imbalance and queue capacity requirements on representative kernel configurations.

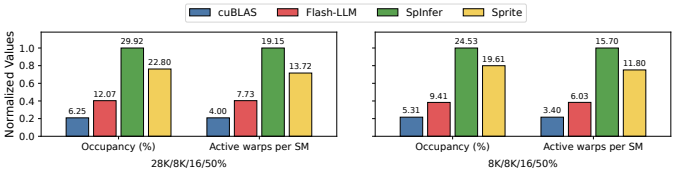


Fig. 16. Occupancy profiling on representative configurations.

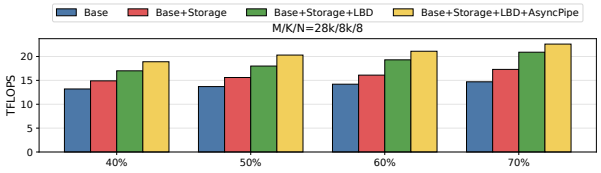


Fig. 17. Kernel performance gain breakdown on A100.

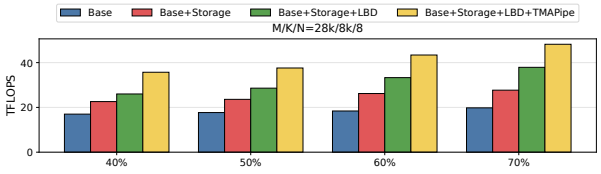


Fig. 18. Kernel performance gain breakdown on H200.

In terms of DRAM reads, both SpInfer and Sprite substantially reduce the memory footprint with bitmap-based compression format, minimizing the amount of data transferred between global memory and SMs. Sprite reads 242.2 MB from DRAM, only slightly more than the 235.9 MB read by SpInfer. This small overhead results from the padding introduced by Sprite's thread-order layout and is limited by the BlockTile-level tile-merging strategy.

Additionally, compared to SpInfer, Sprite increases memory-bandwidth utilization from 49.4% to 87.2%. At the same time, compute-throughput utilization decreases from 71.0% to 48.1%. The simultaneous increase in memory-bandwidth utilization and decrease in compute-throughput indicates that Sprite removes much of the scalar decoding bottleneck in SpInfer.

B. Kernel Ablation Study

Performance gain breakdown. To quantify the impact of key optimizations in Sprite SpMM kernel, we break down the performance gain across multiple sparsity levels in Figure 17 and 18. We take SpInfer as the base kernel, which faces bitmap decoding bottleneck. We progressively integrate Sprite's three key components: ❶ ToSparsE compression format(Storage); ❷ Lightweight Bitmap decoding algorithm (LBD); ❸ Asynchronous Software Pipeline (AsyncPipe) on A100 GPU; TMA and warp-specialization (TMAPipe) on H200 GPU. Each component provides a consistent incremental improvement. On H200 GPU, storage and decoding designs achieve 1.52-1.97× speedups over SpInfer, while TMA-based warp-specialized pipelining provides an additional 1.26-1.38× speedup.

Padding overhead analysis. Figure 19 compares the memory footprints of dense cuBLAS, SpInfer, Sprite without tile merging, and Sprite with tile merging. Applying padding independently at fine-grained tile boundaries substantially increases storage consumption and offsets much of the compression benefit, particularly at lower sparsity levels. Tile merging amortizes imbalance over the reduction dimension and consistently reduces the memory footprint.

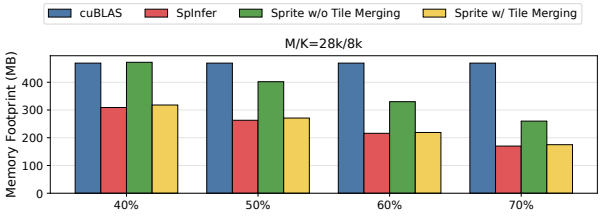


Fig. 19. Ablation analysis of kernel memory footprint.

Figure 20 compares the storage overhead and compression ratios of Sprite and SpInfer across different sparsity levels. Overall, Sprite achieves compression efficiency comparable to SpInfer, with only a modest storage overhead ranging from 1.90% to 4.00%. As sparsity increases from 30% to 70%, Sprite's compression ratio improves from 1.28× to 2.63×, closely following SpInfer's improvement from 1.31× to 2.73×. At 50% sparsity, Sprite requires 260.6 MiB of storage, only 2.81% more than SpInfer (253.5 MiB). The Value memory footprint increases from 224.4 to 231.3 MiB, which stems from the padding overhead introduced by the thread-order format. The storage for Metadata increases from 29.1 to 29.3 MiB, since Sprite needs to store the per-KTile loading length.

C. End-to-end LLM Evaluation

Models. We evaluate the end-to-end inference performance of Sprite on Qwen3-32B and LLaMA3.1-70B models. To preserve model quality, we leverage ATP [26] pruning method to prune the evaluated models at 60% sparsity level. We report the 5-shot accuracy on the MMLU [22] dataset for dense and sparse models. Specifically, the accuracy for Qwen3-32B decreases from 82.4% to 77.6%, while LLaMA3.1-70B's accuracy declines from 83.6% to 80.9%.

Settings and Baselines. We benchmark with batch sizes of 8, 16 and 32, input sequence length of 128 and varied output sequence lengths of 128, 256, 512 and 1024 tokens to represent different inference workloads. We compare Sprite against Flash-LLM, SpInfer as well as the dense baseline. We integrate the evaluated SpMM kernels into vLLM [29], a state-of-the-art LLM inference and serving framework, which enables high-efficiency distributed inference. We use the metric of generated tokens per second to measure the inference throughput.

Preprocessing time. The offline model conversion for Qwen3-32B and LLaMA3.1-70B takes 771s and 1473s. On

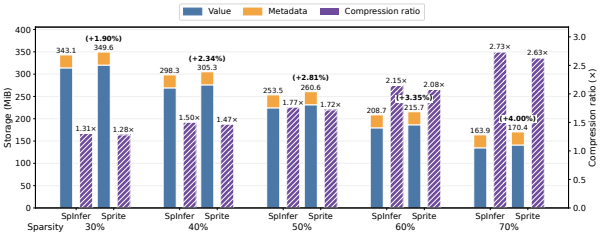


Fig. 20. Memory footprint breakdown and compression ratio comparison.

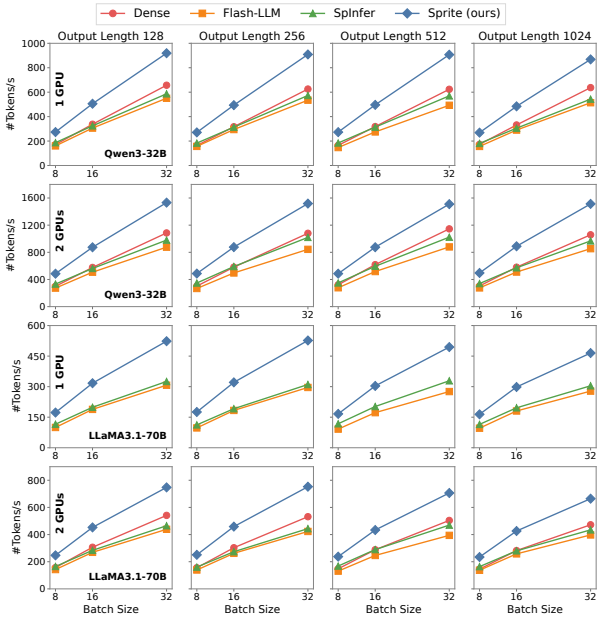


Fig. 21. End-to-end inference performance of Qwen3-32B and LLaMA3.1-70B on A100 GPUs.

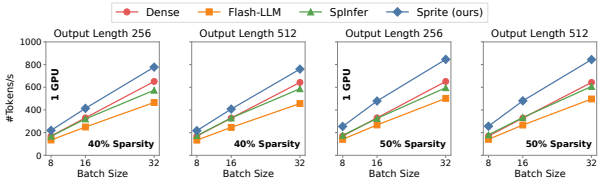


Fig. 22. End-to-end inference performance of Qwen3-32B on A100 at different sparsity levels.

1 H200 GPU, checkpoint-loading time for converted Qwen3-32B and LLaMA3.1-70B are 5.29s and 12.26s. On 2 H200 GPUs, checkpoint-loading time for converted Qwen3-32B and LLaMA3.1-70B are 6.79s and 15.54s.

Results. Figure 21 and 23 show the end-to-end inference throughput of Sprite and the baselines on A100 and H200 GPUs, respectively. Sprite consistently delivers the highest throughput across the evaluated models, GPU counts, batch sizes, and output lengths. On A100, Sprite achieves average speedups of 1.52×, 1.74× and 1.57× compared to vLLM, Flash-LLM and SpInfer, respectively. On H200, Sprite outperforms vLLM, Flash-LLM and SpInfer by 1.46×, 1.72× and 2.05× on average.

In addition to improving throughput, Sprite reduces the memory required to store and execute the pruned models. For example, when performing Qwen3-32B inference with batch size of 8, input length of 128, and output length of 512, Sprite's compressed model consumes 36.8 GB memory, reduces memory requirement by 46.8% compared to 69.2 GB for the dense baseline. Furthermore, in memory-constrained scenarios, Sprite demonstrates its effectiveness in practical deployment for the dense model that exceeds the GPU memory limit. For example, since the memory requirement of LLaMA3.1-70B model surpasses the A100 GPU's capacity, the dense baseline encounters Out-Of-Memory (OOM) errors, while the 60% pruned model can be deployed with Sprite inference framework.

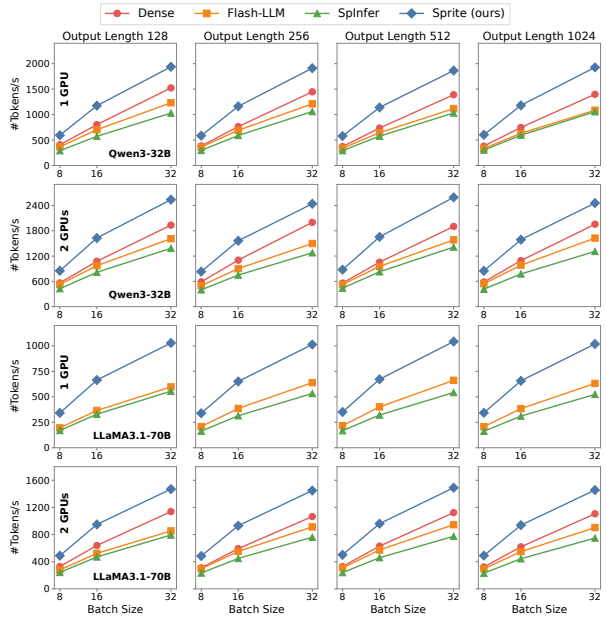


Fig. 23. End-to-end inference performance of Qwen3-32B and LLaMA3.1-70B on H200 GPUs.

We report the throughput across multiple sparsity levels in Figure 22. At 40% and 50% sparsity, the accuracy of Qwen3-32B decreases by 0.46 and 1.37 percentage point, achieving 1.28× and 1.41× end-to-end speedup, respectively, which characterizes the throughput-quality tradeoff.

Breakdown analysis. To understand the source of the end-to-end performance gains, we conduct the end-to-end breakdown of system execution with batch size of 8, input length of 128, and output length of 512, using NSight System, as shown in Figure 24. Across both single- and multi-GPU scenarios, matrix multiplication kernels account for the majority of inference latency. Specifically, SpMM is the primary computational bottleneck in sparse frameworks, whereas GEMM dominates the execution time of dense baselines. Under the same experimental settings, Sprite achieves substantially lower

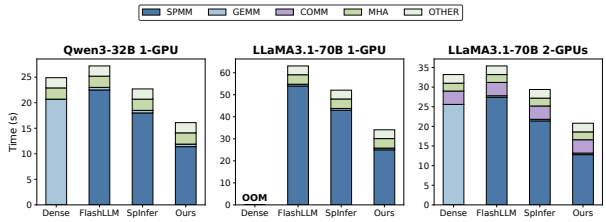


Fig. 24. Breakdown of end-to-end inference time on A100 GPUs. COMM: Inter-GPU Communication. MHA: Multi-Head Attention.

SpMM latency than both Flash-LLM and SpInfer, while also outperforming the GEMM kernels used by vLLM. These kernel-level efficiency gains translate directly into the end-to-end inference speedups delivered by Sprite.

VI. Related Work

Model Compression Technique. Quantization reduces numerical precision of model parameters and activations with low-bit formats. Prior work has improved both quantization algorithms, including post-training quantization [7], [14], [28], [32], [57] and quantization-aware training [6], [9], [36], [58], as well as system-level optimizations [15], [34], [49], [64], enabling broad practical application. On the other hand, pruning selectively removes less important weights, introducing sparsity in weight matrices. Existing pruning methods can be divided into structured [16], [38], [55], [63], semi-structured [12], [66], [68], and unstructured pruning [5], [13], [46], [69]. Structured and semi-structured pruning provide hardware-friendly sparsity patterns but incur larger accuracy loss. In contrast, unstructured pruning removes arbitrary individual weights, providing the highest flexibility and generally preserving model quality better at the same sparsity level.

High Performance SpMM Kernel. Research on GPU-based SpMM spans a wide range of kernel designs and sparse representations, implementing specialized systems tailored to different sparsity regimes. Previous works [8], [10], [23]–[25], [39], [42], [45], [51], [52], [56], [67] have focused on accelerating SpMM for scientific computing and GNN workloads at extreme sparsity levels (>90%). Although not compatible with low-level sparse LLMs, these methods propose advanced optimization techniques for SpMM on both CUDA Cores and Tensor Cores. For deep learning workloads, many works have focused on a high sparsity range (70%-90%) to accelerate sparse model inference. SparTA [70] partitions matrices into 2:4 and unstructured sparse components, leveraging both sparse Tensor Cores and CUDA Cores. Auto-tuners such as SparseTIR [60], LO-SpMM [33] and CROSS [35] efficiently search for high performance kernel implementations. Later approaches such as Flash-LLM [54] and GeneralSparse [50] target the memory-bound inference hotspot in LLMs, but overlook the typical sparsity range (30%-70%) for LLM pruning algorithms. More recent methods including SpInfer [11] and Coruscant [27] reveal the indexing overhead at low-level sparsity for sparse LLMs and leverage bitmap-based formats to achieve effective compression. While achieving noticeable speedup, these methods still struggle with substantial computation overheads in decoding irregular non-zero elements. In contrast, our Sprite adopts a thread-order storage format that reduces decoding operations, enabling pipeline-friendly execution while addressing both SpInfer's decoding overhead and Flash-LLM's data movement overhead.

LLM Inference System Optimization. Improving the efficiency of LLM inference has become a critical research focus due to the growing computational demands. Significant efforts have focused on improving the efficiency of LLM serving systems through better scheduling, memory management, and resource disaggregation. Orca [62] introduces iteration-level scheduling to enable concurrent execution across different request stages, whereas vLLM [29] optimizes memory utilization through dynamic KV-cache management. FlexGen [44], Sarathi-Serve [2], and FastServe [53] further employ advanced scheduling mechanisms to efficiently distribute workloads under constrained hardware resources. Recent optimizations [43], [71] decouple the prefill and decoding phases, giving rise to disaggregated serving architectures. By integrating seamlessly with existing LLM acceleration frameworks, our work enhances the overall efficiency of sparsified LLM inference.

VII. Limitation and Discussion

Sprite primarily targets memory-bound SpMM workloads by reducing memory traffic and decoding overhead. However, Sprite still relies on dense TC MMA instructions and does not skip the computations associated with zero-valued elements. Consequently, for compute-bound workloads (e.g., large N), it cannot achieve performance gains, as the computational cost remains the same to dense matrix multiplication. Incorporating Sparse Tensor Core into SpMM kernels for computation skipping is a promising direction for future work.

While currently optimized for NVIDIA FP16 fragment and shared-memory organization, Sprite can be adapted to broader architectures. For Hopper WGMMA, Sprite can be constructed according to the WGMMA's warpgroup-level operand mapping. For Blackwell TCGen05, the decoding algorithm needs to reconstruct dense operands in shared memory. For AMD Wave64, Sprite requires adaptations to the 64-lane wavefront organization, LDS bank layout, and MFMA operand mapping.

Sprite can be extended to low-precision formats by adopting group-wise sparsity, where adjacent elements are pruned together. This sparsity pattern enables group-level bitmap encoding, which maintains high compression ratio and aligns well with the packed representation of low-precision formats, such as MXFP8 and NVFP4.

VIII. Conclusion

In this paper, we present Sprite, a novel framework that bridges the gap in SpMM kernel to accelerate sparse LLM inference. Sprite introduces Thread-Order Sparse Encoding (ToSparsE), a hardware-aware sparse compression format that arranges irregular non-zero elements into a thread-order sequence, which unlocks the opportunity to reduce bitmap decoding overhead with regular memory layout. Furthermore, Sprite incorporates a buffer-based decoding algorithm and tailored optimizations for the SpMM kernel, further enhancing computational efficiency. The reduction of decoding overhead addresses a critical gap in bitmap-based compression technique, strengthening the benefits of current sparse inference techniques. Our evaluation demonstrates that Sprite consistently surpasses state-of-the-art SpMM kernels across various sparsity levels, and delivers substantial improvements in end-to-end inference speed.

REFERENCES

[1] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[2] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. Gulavani, A. Tumanov, and R. Ramjee, "Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}," in *18th USENIX symposium on operating systems design and implementation (OSDI 24)*, 2024, pp. 117–134.

[3] L. A. Agrawal, S. Tan, D. Soylu, N. Ziems, R. Khare, K. Opsahl-Ong, A. Singhvi, H. Shandilya, M. J. Ryan, M. Jiang, C. Potts, K. Sen, A. Dimakis, I. Stoica, D. Klein, M. Zaharia, and O. Khattab, "GEPA: Reflective prompt evolution can outperform reinforcement learning," in *The Fourteenth International Conference on Learning Representations*, 2026.

[4] S. Ashkboos, M. L. Croci, M. G. D. Nascimento, T. Hoefler, and J. Hensman, "Slicegpt: Compress large language models by deleting rows and columns," in *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*.

[5] G. Bai, Y. Li, C. Ling, K. Kim, and L. Zhao, "Sparsellm: Towards global pruning for pre-trained language models," *Advances in neural information processing systems*, 2024.

[6] M. Chen, W. Shao, P. Xu, J. Wang, P. Gao, K. Zhang, and P. Luo, "Efficientqat: Efficient quantization-aware training for large language models," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 10 081–10 100.

[7] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "Gpt3. int8 (): 8-bit matrix multiplication for transformers at scale," *Advances in Neural Information Processing Systems*, vol. 35, pp. 30 318–30 332, 2022.

[8] Y. Dong, Z. Shen, W. Jiang, Z. Liu, Y. Xu, B. He, R. Zheng, and H. Jin, "Bridging the gap between unstructured spmm and structured sparse tensor cores," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2025, pp. 645–660.

[9] D. Du, Y. Zhang, S. Cao, J. Guo, T. Cao, X. Chu, and N. Xu, "Bitdistiller: Unleashing the potential of sub-4-bit llms via self-distillation," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 102–116.

[10] R. Fan, W. Wang, and X. Chu, "Dtc-spmm: Bridging the gap in accelerating general sparse matrix multiplication with tensor cores," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 253–267.

[11] R. Fan, X. Yu, P. Dong, Z. Li, G. Gong, Q. Wang, W. Wang, and X. Chu, "Spinfer: Leveraging low-level sparsity for efficient large language model inference on gpus," in *Proceedings of the Twentieth European Conference on Computer Systems*, 2025, pp. 243–260.

[12] G. Fang, H. Yin, S. Muralidharan, G. Heinrich, J. Pool, J. Kautz, P. Molchanov, and X. Wang, "Maskllm: Learnable semi-structured sparsity for large language models," *Advances in neural information processing systems*, 2024.

[13] E. Frantar and D. Alistarh, "Sparsegpt: Massive language models can be accurately pruned in one-shot," in *International Conference on Machine Learning*. PMLR, 2023, pp. 10 323–10 337.

[14] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "Gptq: Accurate post-training quantization for generative pre-trained transformers," *arXiv preprint arXiv:2210.17323*, 2022.

[15] E. Frantar, R. L. Castro, J. Chen, T. Hoefler, and D. Alistarh, "Marlin: Mixed-precision auto-regressive parallel inference on large language models," in *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, 2025, pp. 239–251.

[16] Y. Fu, Z. Yu, J. Li, J. Qian, Y. Zhang, X. Yuan, D. Shi, R. Yakunin, and Y. C. Lin, "Amoeballm: Constructing any-shape large language models for efficient and instant deployment," *Advances in Neural Information Processing Systems*, vol. 37, pp. 78 299–78 319, 2024.

[17] T. Gale, M. Zaharia, C. Young, and E. Elsen, "Sparse gpu kernels for deep learning," in *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2020, pp. 1–14.

[18] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[19] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi *et al.*, "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.

[20] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.

[21] Y. He, X. Zhang, and J. Sun, "Channel pruning for accelerating very deep neural networks," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 1389–1397.

[22] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, "Measuring massive multitask language understanding," in *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7*, 2021.

[23] C. Hong, A. Sukumaran-Rajam, I. Nisa, K. Singh, and P. Sadayappan, "Adaptive sparse tiling for sparse matrix multiplication," in *Proceedings of the 24th Symposium on Principles and Practice of Parallel Programming*, 2019, pp. 300–314.

[24] J. Hu, H. Luo, H. Jiang, M. Casas, K. Li, and C. Liu, "Swift: High-performance sparse-dense matrix multiplication on gpus," in *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2026, pp. 1–16.

[25] Z. Hu, J. Sun, and G. Sun, "Gnnpilot: A holistic framework for high-performance graph neural network computations on gpus," *ACM Transactions on Architecture and Code Optimization*, vol. 22, no. 2, pp. 1–26, 2025.

[26] W. Huang, Y. Zhang, X. Zheng, F. Chao, and R. Ji, "Determining layer-wise sparsity for large language models through a theoretical perspective," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267. PMLR, 13–19 Jul 2025, pp. 26 175–26 201.

[27] D. Joo, H. Hosseini, R. Hadidi, and B. Asgari, "Coruscant: Co-designing gpu kernel and sparse tensor core to advocate unstructured sparsity in efficient llm inference," in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, 2025, pp. 232–245.

[28] S. Kim, C. R. C. Hooper, A. Gholami, Z. Dong, X. Li, S. Shen, M. W. Mahoney, and K. Keutzer, "Squeezellm: Dense-and-sparse quantization," in *International Conference on Machine Learning*. PMLR, 2024, pp. 23 901–23 923.

[29] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th symposium on operating systems principles*, 2023, pp. 611–626.

[30] J. Li, T. Tang, W. X. Zhao, J.-Y. Nie, and J.-R. Wen, "Pre-trained language models for text generation: A survey," *ACM Computing Surveys*, vol. 56, no. 9, pp. 1–39, 2024.

[31] L. Li, Y. Zhang, and L. Chen, "Prompt distillation for efficient llm-based recommendation," in *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, 2023, pp. 1348–1357.

[32] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "Awq: Activation-aware weight quantization for on-device llm compression and acceleration," *Proceedings of machine learning and systems*, vol. 6, pp. 87–100, 2024.

[33] J. Lin, J. Sun, X. Shi, H. Zhang, X. Yu, X. Wang, J. Yao, and G. Sun, "Lo-spmm: Low-cost search for high-performance spmm kernels on gpus," *ACM Transactions on Architecture and Code Optimization*, vol. 21, no. 4, pp. 1–25, 2024.

[34] Y. Lin, H. Tang, S. Yang, Z. Zhang, G. Xiao, C. Gan, and S. Han, "Qserve: W4a8kv4 quantization and system co-design for efficient llm serving," *Proceedings of Machine Learning and Systems*, vol. 7, 2025.

[35] F. Liu, S. Huang, N. Yang, Z. Wang, H. Li, and L. Jiang, "Cross: Compiler-driven optimization of sparse dnns using sparse/dense computation kernels," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 963–976.

[36] Z. Liu, B. Oguz, C. Zhao, E. Chang, P. Stock, Y. Mehdad, Y. Shi, R. Krishnamoorthi, and V. Chandra, "Llm-qat: Data-free quantization aware training for large language models," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 467–484.

[37] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, and C. Zhang, "Learning efficient convolutional networks through network slimming," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 2736–2744.

[38] X. Ma, G. Fang, and X. Wang, "Llm-pruner: On the structural pruning of large language models," *Advances in neural information processing systems*, vol. 36, pp. 21 702–21 720, 2023.

[39] P. Okanovic, G. Kwasniewski, P. S. Labini, M. Besta, F. Vella, and T. Hoefler, "High performance unstructured spmm computation using tensor cores," in *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2024, pp. 1–14.

[40] H. Pan, C. Wang, M. Qiu, Y. Zhang, Y. Li, and J. Huang, "Meta-kd: A meta knowledge distillation framework for language model compression across domains," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, 2021, pp. 3026–3036.

[41] Z. Pan, H. Luo, M. Li, and H. Liu, "Chain-of-action: Faithful and multimodal question answering through large language models," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 63 920–63 939.

[42] M. Pang, X. Fei, P. Qu, Y. Zhang, and Z. Li, "A row decomposition-based approach for sparse matrix multiplication on gpus," in *Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, 2024, pp. 377–389.

[43] R. Qin, Z. Li, W. He, J. Cui, F. Ren, M. Zhang, Y. Wu, W. Zheng, and X. Xu, "Mooncake: Trading more storage for less computation—a {KVCache-centric} architecture for serving {LLM} chatbot," in *23rd USENIX conference on file and storage technologies (FAST 25)*, 2025, pp. 155–170.

[44] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, "Flexgen: High-throughput generative inference of large language models with a single gpu," in *International Conference on Machine Learning*. PMLR, 2023, pp. 31 094–31 116.

[45] J. Shi, S. Li, Y. Xu, R. Fu, X. Wang, and T. Wu, "Flashsparse: Minimizing computation redundancy for fast sparse matrix multiplications on tensor cores," in *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, 2025, pp. 312–325.

[46] M. Sun, Z. Liu, A. Bair, and J. Z. Kolter, "A simple and effective pruning approach for large language models," in *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*.

[47] S. Sun, Z. Gan, Y. Fang, Y. Cheng, S. Wang, and J. Liu, "Contrastive distillation on intermediate representations for language model compression," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 498–508.

[48] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[49] L. Wang, L. Ma, S. Cao, Q. Zhang, J. Xue, Y. Shi, N. Zheng, Z. Miao, F. Yang, T. Cao *et al.*, "Ladder: Enabling efficient {Low-Precision} deep learning computing through hardware-aware tensor transformation," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 307–323.

[50] Y. Wang, X. Guo, J. Xiao, D. Chen, and G. Tan, "{GeneralSparse}: Bridging the gap in {SpMM} for pruned large language model inference on {GPUs}," in *2025 USENIX Annual Technical Conference (USENIX ATC 25)*, 2025, pp. 417–432.

[51] Y. Wang, B. Feng, G. Li, S. Li, L. Deng, Y. Xie, and Y. Ding, "{GNNAdvisor}: An adaptive and efficient runtime system for {GNN} acceleration on {GPUs}," in *15th USENIX symposium on operating systems design and implementation (OSDI 21)*, 2021, pp. 515–531.

[52] Y. Wang, B. Feng, Z. Wang, G. Huang, and Y. Ding, "{TC-GNN}: Bridging sparse {GNN} computation and dense tensor cores on {GPUs}," in *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, 2023, pp. 149–164.

[53] B. Wu, Y. Zhong, Z. Zhang, S. Liu, F. Liu, Y. Sun, G. Huang, X. Liu, and X. Jin, "{FastServe}:{Iteration-Level} preemptive scheduling for large language model inference," in *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*, 2026, pp. 57–74.

[54] H. Xia, Z. Zheng, Y. Li, D. Zhuang, Z. Zhou, X. Qiu, Y. Li, W. Lin, and S. L. Song, "Flash-llm: Enabling cost-effective and highly-efficient large generative model inference with unstructured sparsity," *Proceedings of the VLDB Endowment*, vol. 17, no. 2, pp. 211–224, 2023.

[55] M. Xia, T. Gao, Z. Zeng, and D. Chen, "Sheared llama: Accelerating language model pre-training via structured pruning," in *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*.

[56] Y. Xia, W. Wang, D. Yang, X. Zhou, and D. Cheng, "Voltrix: Sparse {Matrix-Matrix} multiplication on tensor cores with asynchronous and balanced kernel optimization," in *2025 USENIX Annual Technical Conference (USENIX ATC 25)*, 2025, pp. 699–714.

[57] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "Smoothquant: Accurate and efficient post-training quantization for large language models," in *International Conference on Machine Learning*. PMLR, 2023, pp. 38 087–38 099.

[58] Y. Xu, X. Han, Z. Yang, S. Wang, Q. Zhu, Z. Liu, W. Liu, and W. Che, "Onebit: Towards extremely low-bit large language models," *Advances in Neural Information Processing Systems*, vol. 37, pp. 66 357–66 382, 2024.

[59] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv *et al.*, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.

[60] Z. Ye, R. Lai, J. Shao, T. Chen, and L. Ceze, "Sparsetir: Composable abstractions for sparse compilation in deep learning," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2023, pp. 660–678.

[61] L. Yin, Y. Wu, Z. Zhang, C.-Y. Hsieh, Y. Wang, Y. Jia, G. Li, A. K. Jaiswal, M. Pechenizkiy, Y. Liang *et al.*, "Outlier weighed layerwise sparsity (owl): A missing secret sauce for pruning llms to high sparsity," in *International Conference on Machine Learning*. PMLR, 2024, pp. 57 101–57 115.

[62] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for {Transformer-Based} generative models," in *16th USENIX symposium on operating systems design and implementation (OSDI 22)*, 2022, pp. 521–538.

[63] M. Zhang, H. Chen, C. Shen, Z. Yang, L. Ou, X. Yu, and B. Zhuang, "Loraprune: Structured pruning meets low-rank parameter-efficient fine-tuning," in *Findings of the Association for Computational Linguistics ACL 2024*, 2024, pp. 3013–3026.

[64] Q. Zhang, M. Tang, M. Zhai, K. Lei, and J. Zhai, "Romeo: Mitigating dual-dimensional outliers with rotated mixed precision quantization," in *Proceedings of the 31st ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, 2026, pp. 273–287.

[65] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin *et al.*, "Opt: Open pre-trained transformer language models," *arXiv preprint arXiv:2205.01068*, 2022.

[66] Y. Zhang, H. Bai, H. Lin, J. Zhao, L. Hou, and C. V. Cannistraci, "Plug-and-play: An efficient post-training pruning method for large language models," in *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*.

[67] H. Zhao, S. Li, J. Wang, C. Zhou, J. Wang, Z. Xin, S. Li, Z. Liang, Z. Pan, F. Liu *et al.*, "Acc-spmm: Accelerating general-purpose sparse matrix-matrix multiplication with gpu tensor cores," in *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, 2025, pp. 326–338.

[68] K. Zhao, T. Yuan, H. Bao, Z. Su, C. Gao, Z. Sun, Z. Liang, L. Jing, and J. Chen, "Beyond 2: 4: exploring v: N: M sparsity for efficient transformer inference on gpus," *arXiv preprint arXiv:2410.16135*, 2024.

[69] P. Zhao, F. Sun, X. Shen, P. Yu, Z. Kong, Y. Wang, and X. Lin, "Pruning foundation models for high accuracy without retraining," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024, pp. 9681–9694.

[70] N. Zheng, B. Lin, Q. Zhang, L. Ma, Y. Yang, F. Yang, Y. Wang, M. Yang, and L. Zhou, "{SparTA}:{Deep-Learning} model sparsity via {Tensor-with-Sparsity-Attribute}," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 213–232.

[71] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "{DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 193–210.